

ENDS

Backend Design Document — v1.0

Companion to: ENDS Game Design Document v1.1

Scope: Server architecture, services, data model, real-time protocol, security, anti-cheat, GDPR, infrastructure, capacity, costs, and backend milestones aligned to the GDD's M0–M5 roadmap.

Audience: Snicla (solo dev building with Claude Code). Written to be handed, section by section, to an AI coding agent as build specs.

Locked upstream decisions this doc inherits: 2D top-down real-time world · landscape · mobile first · energy + timers · simple stat combat · **no open-world PvP** (NPC threat + CoC-style async wars with optional live defence).

1. Purpose & Scope

This document specifies everything on the other side of the socket: what the server owns, how the client talks to it, how we stop cheaters, how we stay legal in the EU/UK, what it runs on, and what it costs. It deliberately repeats one message throughout: **the server is the game; the client is a very pretty liar**. Every number that matters — cash, loot, XP, war stripes — is computed and stored server-side. The phone renders the story.

Out of scope: client engine internals, art pipeline, store/publishing ops (flagged where they touch the backend), monetisation systems (constraint inherited from GDD §22.4: the economy must stand without selling power).

2. Architecture Doctrine — Why ENDS Is Backend-Cheap

The single most important fact about this backend: **the PvE decision (GDD §11, §14) removed the hard problems**. No open-world PvP means:

- **No competitive physics sync.** Player positions in districts are *social presence*, not combat state. If your sprite is 40 cm off on my screen, nothing unfair happens. So: 10 Hz position echo with interpolation, no rollback, no lockstep, no tick-perfect simulation.
- **All combat is player-vs-server.** NPC fights, raids, hit squads — the server resolves the GDD §10 formulas and the client animates the result. Latency tolerance: hundreds of milliseconds, invisible.
- **Raids are small instanced sims** (1 attacker + up to 3 NPC crew vs an AI defence; up to a handful of humans if live defence triggers). Never 50-player battles.
- **Wars are asynchronous state machines** resolved by scheduled jobs. Clash of Clans runs on this shape at planetary scale for a reason: it is cheap, robust and timezone-proof.
- **The bulk of the game is meta:** timers, ledgers, inventories, territory records, notifications. That is CRUD with strong transactions — solved technology.

Consequence: a solo developer can genuinely run this. The launch topology is **one application node + one Postgres** on a mid-range EU VPS, and the scale path is boring (add nodes, split the DB). The doctrine in one line: *authoritative, asynchronous by default, real-time only where humans can see each other*.

3. High-Level Architecture

```

[ Game client: iOS / Android ]
|
|   HTTPS RPC (meta:      \ WebSocket (real-time:
|   auth, collect, crimes, \ district presence, raids,
|   economy, wars, config) \ heists, live defence, chat)
|
v
[ Caddy - TLS termination / reverse proxy ]
v
[ Nakama node(s) ]
- Authoritative match handlers: district rooms, raid instances, heist instances
- Runtime modules (TypeScript): economy, crime resolution, timers, wars,
  lines, notifications, telemetry ingest, admin RPCs
- Built-ins used: auth/sessions, storage engine, groups (=Firms), friends,
  chat channels, leaderboards, tournaments, presence/streams, matchmaker
v
[ PostgreSQL 16 ] <- single source of truth (ledger, players, territory, wars...)
[ Object storage ] <- client asset bundles, backups (Hetzner Storage Box / S3-compatible)
[ Metabase ]      <- dashboards on read replica / same DB at launch
[ FCM + APNs ]    <- push notification delivery

```

Two transport lanes, one brain:

- **Meta lane (request/response RPC):** everything that is not "other humans moving on my screen". Stateless per call, transactional in the DB.
- **Real-time lane (WebSocket):** district presence, instanced co-op (raids/heists/live defence), chat. Nakama multiplexes both over one socket once connected; the distinction is logical.

4. Tech Stack

4.1 Recommendation

Layer	Choice	Why
Game server framework	Nakama 3.x (open source, self-hosted)	Auth, storage, groups (= Firms) , friends, chat, leaderboards, tournaments, presence, matchmaker and authoritative real-time matches out of the box — six months of solo work you do not have to do. Apache-2.0, no per-CCU licence. Official Godot / Unity / JS SDKs.
Server logic language	TypeScript (Nakama runtime modules)	Closest to your existing web stack; first-class Claude Code territory; Nakama also allows Go modules later for hot paths.
Database	PostgreSQL 16	Nakama's native store; also holds our custom relational tables (ledger, wars, territory). One database to operate.
Real-time transport	WebSocket, JSON payloads at first	Debuggable; switch hot messages to binary later only if profiling says so (\$7.5).
Reverse proxy / TLS	Caddy	Zero-config certificates, sane defaults.
Dashboards	Metabase (self-hosted)	SQL-first KPI dashboards (D1/D7, sink ratio, funnels) without a SaaS bill.
Push	FCM (Android) + APNs (iOS), called from a Nakama module	Free, direct, no OneSignal dependency.
Hosting	Hetzner (EU — Falkenstein/Nuremberg)	Best price/perf in Europe; UK round-trip 25–40 ms — irrelevant for this design's latency needs. GDPR-friendly.

4.2 Alternatives Considered

Option	Verdict
Node.js + Colyseus + Fastify + Postgres + Redis	Excellent rooms model, but you rebuild auth, social graph, groups, leaderboards, push plumbing yourself. Choose only if Nakama's structure feels too opinionated after the M1 spike.

Option	Verdict
Managed: PlayFab (meta) + Photon (real-time)	Fastest week one, then vendor lock-in, per-CCU pricing, and two dashboards forever. Rejected for a game whose moat is a custom economy.
Firebase/Supabase only	No authoritative real-time match model; you would hand-roll the hardest part. Rejected.
Elixir/Phoenix	Technically superb for this shape; new language + ecosystem for a solo dev under time pressure. Rejected on pragmatics.

4.3 Client Engine (flag — owned by the front-end phase, constrains SDK choice)

Recommendation: Godot 4.x. Free, excellent 2D (normal-mapped lights for the sodium-lamp look, GDD §20.3), small mobile builds, official Nakama SDK, very Claude-Code-workable. Unity is the fallback if asset-store middleware becomes decisive. The protocol in §7 is engine-agnostic either way. **Decision needed before M1.**

5. Core Services

Each service below is a Nakama runtime module (TS) plus its tables/collections. They are listed in rough build order.

5.1 Auth & Accounts

- Device auth on first launch (instant play — FTUE must not show a login wall, GDD §18.5), silent account creation; email/social link offered later ("secure your Ps") for recovery.
- Nakama sessions: JWT, 15-min tokens + refresh; socket auth on the same session.
- One character per account at launch. Display name uniqueness enforced at claim time with the moderation filter (§5.20).
- Bans/suspensions: account-level flags checked at session refresh; device-hash soft-ban list for repeat offenders.

5.2 Player Profile & Stats

- Storage: `players` (identity, rank, rep, origin, city, tutorial step), `player_stats` (STR/TGH/SPD/SLK + training queue), `crime_skills` (per-track XP/level), wardrobe/inventory (§5.4), settings (notification categories, locale).
- All progression writes happen server-side as side-effects of resolved actions — there is no "set my XP" surface anywhere.

5.3 Resource & Timer Engine (the heartbeat)

Pattern: lazy evaluation. Nothing ticks per player. Every regenerating value is stored as `(value_at, updated_at, rate, cap)` and computed on read:

```
function current(r: Regen, now: number): number {
  return Math.min(r.cap, r.valueAt + r.rate * (now - r.updatedAt));
}
// spend() = recompute -> check >= cost -> write new (value, now) in the same tx
```

- Applies to Energy, Nerve, heat decay, business/territory income accrual (accrue-to-cap on read), gym queues, crew training, laundering batches, line cycles, travel, jail/hospital, shields.
- **Scheduled jobs exist only where the world must act without the player:** war resolution, season rollover, notification scans (§5.16), runner round-trips completing into "arrived" state, invariant audits. All jobs idempotent and re-runnable.
- Server clock is the only clock. Client renders countdowns from server-issued `ends_at` timestamps; client time is never trusted. (Bonus: the GDD's "night = real UK time" rule is trivially server-truthful.)

5.4 Economy & Ledger

The most protected service in the game.

- **Double-entry style append-only ledger:** `ledger(id, player_id, currency ['dirty'|'clean'], delta, reason, ref_type, ref_id, created_at)`. Cached balances on the player row, recomputed-and-compared by a nightly audit job; any mismatch pages the admin (you).
- Every mutation (crime payout, wage, wash, bail, raid loss, market sale) is **one DB transaction** touching ledger + inventory + whatever else atomically. No partial states, ever.
- Laundering: a `wash_batches` queue (`front_id, dirty_in, fee_pct, done_at`) — lazy-completed on read, claimable transactionally.
- Global economy telemetry falls out of the ledger for free: minted vs sunk per reason per day = the live **sink-ratio dashboard** promised in GDD §15.3, from M2.
- Marketplace (§5.14) moves money only between players + tax sink; it never mints.

5.5 Crime Resolution (server-rolled, client-performed)

The anti-cheat spine for the game's most frequent action:

1. `crime_start(type, target_ref)` -> server validates: player in the right district instance, Nerve $\geq$ cost, cooldowns, rank/skill gates -> deducts Nerve, creates `crime_sessions` row (`id, seed, params, started_at, min_ms, max_ms`) -> returns session + params (e.g. mark value band, alarm timer length).
 2. Client plays the 10–60 s minigame **as theatre**.
 3. `crime_finish(session_id, metrics)` -> server checks: single use, duration within `[min_ms, max_ms]`, metrics plausible (taps within human ranges, movement continuity) -> rolls the outcome **from the stored seed** through the GDD Appendix A.1 tables (base band / good / crit 8% / jackpot 2%, pity floor §18.2) -> applies loot, XP, heat in one transaction -> returns the result payload the reveal ceremony renders.
 4. Implausible metrics never hard-fail in the moment (false positives are poison): the action succeeds at floor value and the account gets a shadow-flag for the review queue (§8.4).
- Rarity, near-miss truth (§18.3) and payouts are therefore **facts decided by the server**, and the reveal is honest by construction.

5.6 District Presence (real-time rooms)

- Each district = an authoritative Nakama match. Capacity 50; instance #2+ spawns on overflow; placement priority firm-mates > friends > fill (social gravity by default).
- Client sends position/intent at **10 Hz**, server clamps against max velocity (+15% tolerance) and teleport checks, echoes at 10 Hz with **interest management**: 16 m grid cells, subscribe to your 3x3 (roughly a screen + margin at landscape zoom). Remote players are interpolated client-side; positions are cosmetic-social by doctrine (§2).
- The match handler also hosts: NPC sim (§5.7), street-event rolls, context-action arbitration (two players tapping the same mark: first-come server-side, loser gets a graceful "someone got there first" — an intentional social spark).
- Idle sockets in a district for 5+ min drop to a cheap "lobby presence" (no position echo) until they move — battery and bandwidth mercy.

5.7 NPC Simulation

- Runs inside each district match at **5 Hz**: patrol FSMs on waypoint graphs (Idle -> Patrol -> Suspicious -> Aggro -> Chase -> Return), vision cones evaluated against present players, faction aggro gated by each player's **beef** value (read-through cache of `beef(player_id, faction_id, value, updated_at)`; lazy decay per GDD §11.2).
- Hit squads: scheduled per-player events (`beef <= -70`) that materialise as a spawn near the player in any contested district — created by a small periodic job, delivered on next district entry so nobody is ambushed inside a menu.

- Street events (GDD §11.5): rolled server-side on district join (~20%), instance-scoped so two players can share the same little story.
- Civilian marks (pickpocket targets etc.) are server-spawned entities with server-known carry values — the client cannot see a mark's wallet unless the skill level says so.

5.8 Combat Resolver

- Pure function of the GDD §10 formulas: inputs (both sides' stats, gear, consumable choices per round, flee attempts), seeded RNG, output (round log + result). Runs server-side for every fight type; the client receives the round log and animates it (this is why "simple combat" and "server authoritative" are best friends).
- PvE fights resolve inside the district match; raid fights inside the raid instance; sparring inside the safe-zone match with zero stakes wiring.

5.9 Raid & Instance Service

- `raid_start(target)` -> validate tickets/eligibility -> **snapshot** the defender's layout + assigned defence crew into the raid row (no mid-raid hot-swaps) -> spawn an instanced authoritative match seeded for determinism -> attacker (+ up to 3 NPC crew) plays the 120-second anatomy (GDD §14.4) against the AI defence.
- **Live defence:** on `raid_start`, publish to the defender's (and firm's) notification stream + push with the bypass-cap flag; a 20-second join window admits defenders into the *same* instance as combatants. Async by default, electric when live — the flagship backend moment of the game.
- Resolution applies transactionally: stash delta (capped 20% dirty, GDD §13.3), stripes, defence-repair timer, morale effects, war scoring if in-war, Feed event, both-side result payloads.
- Heists reuse the same instance machinery with 2–4 human slots and role gating; the Lookout role subscribes to the instance from *outside* the scene (phone-only overwatch, GDD §7.4) — technically just another participant with a restricted view.

5.10 War Service

- State machine per war: declared -> prep(24h) -> active(48h) -> resolving -> closed, all transitions by idempotent scheduled jobs.
- Matchmaking guardrails at declaration: power rating = $f(\text{member count, avg rank, defence ratings, trophy history})$, $\pm 30\%$ window, target-not-already-at-war, 14-day new-firm immunity (GDD §14.3).
- Tickets: 5 raids/side/day stored and enforced server-side; stripes ledgered per raid; war chest escrowed at declaration.
- Resolution job: tally stripes, transfer/hold postcode, split chest 60/40, apply 3-day shields, emit ceremonies + Feed + notifications. Every step written so a crashed job can re-run safely.

5.11 Firms & Social Graph

- Firms map 1:1 onto **Nakama groups**: creation fee check, 30-member cap, role tiers (Top Boy / Elder / Soldier / Yout) via group metadata + edge roles.
- Firm bank = a ledger account owned by the group (same ledger table, `owner_type='firm'`); contributions and territory cuts are ordinary ledger transactions with full audit.
- Perk tree = versioned config purchases stored on the group; effects read by other services (defence bonus, line yield, +war ticket).
- Friends, blocks and "vouched" mentorship edges use Nakama's social graph as-is.

5.12 Chat & Moderation

- Nakama chat channels: firm, war (auto-created per war, auto-archived at close), city (rate-limited: 1 msg/10 s/player), DMs.
- Every message passes a server filter chain before fan-out: banned-terms list, link stripping in city chat, per-player mute/ban flags, report attachment ids. Reported messages are retained 30 days for review then purged (§9).

- No voice at launch (GDD §17.2) — a backend blessing: text moderation only.

5.13 Feed (Flexta)

- Event-outbox pattern: gameplay services emit `feed_events(player_id, type, payload, visibility)` inside their transactions; a fan-out worker builds per-follower timelines (or, at launch scale, timelines are just indexed queries — fan-out-on-read is fine below ~50k DAU).
- Reactions are cheap counters with one-per-player uniqueness; reaction rep trickle capped per post (anti-farm).
- Opt-outs per event type honoured at emit time, not render time — unposted means unposted.

5.14 Marketplace (Shifted)

- `listings(id, seller, item_ref, price, tax_pct, state)` with **escrow semantics**: item leaves seller inventory at listing time; buyer purchase is a single transaction (ledger transfer + tax sink + item grant + listing close). No trade windows, no duping surface.
- Price floors/ceilings per item class from config to slow week-one market manipulation; anomaly alerts on outlier trades (§8.4).

5.15 County Lines Service

- `lines(id, owner, town, level, stock, demand_cache, pressure, state) + runs(id, line_id, runner_id, type[auto|escorted], departs, returns, cargo, outcome).`
- Auto-run lifecycle by lazy evaluation + a small completion job (returns land even if the owner is offline — arriving Ps are the notification). Bust rolls are server-side at completion time using pressure/greed inputs (GDD §9.2), never client-visible in advance.
- Escorted runs create a **convoy encounter** instance on the route at a server-chosen window; interception eligibility (rival intel) is computed from beef/war state. The +40% yield / interception risk trade sits entirely in server math.

5.16 Notification Service

The retention engine, enforced server-side so the caps are real (GDD §18.6):

- `notif_state(player_id, day_count, last_sent, quiet_from, quiet_to, category_optouts)`; every candidate notification passes: category opt-out -> quiet hours (player tz) -> daily cap 4 -> priority arbitration. **Raid-alert carries a bypass-cap flag** and is the only one that does.
- Delivery: FCM/APNs tokens per device; deep links (`ends://raid/{id}`, `ends://war/{id}`); copy rotated server-side from pools of 10+ per event so the tray never goes stale.
- Scheduled scans compute "energy full / income capped / streak at risk" moments from the lazy-timer fields — precomputed into a due-queue, drained every minute.

5.17 Leaderboards & Seasons

- Nakama leaderboards for the weekly + seasonal boards (rep, stripes, books, line volume, defence streak), city-scoped by board id.
- Season service: 8-week clock; rollover job executes in order: freeze boards -> grant rewards -> archive war history -> **soft map reset** (return ~50% of player postcodes to NPC factions per GDD §14.6 rules) -> reset seasonal boards -> emit season-start events. Rehearsed on staging before every live rollover, because this job touches everything.

5.18 Live Ops & Remote Config

- **All tuning is data.** GDD Appendix A tables (crime payouts, timers, defence costs, formulas' constants) live as versioned JSON documents in storage; services read through a cached config accessor; hot-reload on version bump — rebalancing without redeploying.
- Feature flags (per-percentage / per-account) for staged rollouts; event scheduler for weekend events (Blackout Storm windows, double-line Sundays) as config, not code.

5.19 Telemetry & Analytics

- Client batches events (≤ 50 events / 30 s) -> `telemetry_ingest` RPC -> append-only partitioned table (`name`, `player_id`, `ts`, `props jsonb`); raw retained 90 days, nightly rollups kept forever (§9).
- Instrumented from M1: FTUE beat funnel (GDD §18.5 minute-marks), retention cohorts, session length/count, crime mix, economy mint/sink by reason, notification open rates, war participation.
- Metabase dashboards on these tables are the KPI source of truth for §22.3 targets. Move raw events to ClickHouse only if Postgres partitions groan (not before ~100k DAU).

5.20 Admin & Moderation Tools

- Admin-only RPC surface + a thin web panel: player lookup (ledger trail, sessions, flags), grant/revoke, ban/suspend, name/tag rejection queue, report queue, shadow-flag review (§8.4), config editor with diff + version history, kill-switches per feature flag.
- Every admin action is itself ledgered (`admin_audit`) — including yours.

6. Data Model (core tables / collections)

Table / collection	Key fields (abridged)	Notes		
players	id, name, rank, rep, origin, city_id, created_at, flags	Nakama user + our extension row		
player_stats	player_id, str, tgh, spd, slk, training_queue	Lazy queue fields		
regen	player_id, kind[energy	nerve	heat], value_at, updated_at, rate, cap	§5.3 pattern
crime_skills	player_id, track, xp, level			
items / inventory	item_id, owner_id, class, rarity, condition, equipped_slot	Wardrobe = equipped subset		
ledger	id, owner_type[player	firm	system], owner_id, currency, delta, reason, ref	Append-only
wash_batches	id, front_id, dirty_in, fee_pct, done_at, claimed			
fronts	id, owner_id, type, tier, district_id			
crew_members	id, owner_id, class, level, traits, morale, assignment, busy_until			
trap_houses	id, owner_id, tier, district_id, layout jsonb, defence_rating	Layout = editor grid		
territories	postcode_id, district_id, holder_type[npc	firm], holder_id, income_rate, held_since		
control_points	id, postcode_id, kind, state			
factions / beef	faction_id meta; beef(player_id, faction_id, value, updated_at)	Lazy decay		
lines / runs	§5.15			
wars	id, attacker_firm, defender_firm, postcode_id, state, chest, prep_ends, ends			
raids	id, war_id?, attacker_id, target_ref, snapshot jsonb, seed, stripes, result			
listings	§5.14			
feed_events / reactions	§5.13			
crime_sessions	§5.5	TTL-cleaned		
notif_state / notif_queue	§5.16			
telemetry_events	partitioned by day			
seasons	id, starts, ends, state			
config_docs	key, version, body jsonb	§5.18		
admin_audit	actor, action, target, before/after			

Rule of thumb for Nakama: social/identity/leaderboards/groups use built-ins; anything with money, territory or war history gets an explicit SQL table with real constraints and indexes. Migrations via a plain SQL migration runner in the repo from day one.

7. Real-Time Protocol

7.1 Transport & Session

Single WebSocket per client (Nakama socket), JSON payloads, gzip permessage-deflate. Reconnect with resume: client rejoins its district match and receives a fresh full snapshot (no delta history needed — presence state is tiny).

7.2 Message Catalogue (core opcodes)

Op	Dir	Payload (abridged)	Rate / notes
join_district	C->S	district_id	Server picks instance
snapshot	S->C	players[], npcs[], events[]	On join / resume
pos	C->S	x, y, facing, moving	10 Hz max, clamped
pos_batch	S->C	cell-local movers	10 Hz, interest-managed
action	C->S	verb, target_ref	Context button; server arbitrates
action_result	S->C	outcome payload	Includes crime session handoffs
npc_delta	S->C	spawns/despawns/state	5 Hz aggregate
fight_log	S->C	round-by-round log	Client animates \$5.8 output
raid_join / raid_state / raid_result	both	instance messages	Instanced matches
defend_prompt	S->C	raid_id, window_ends	Live-defence 20 s window
chat_msg	both	channel, text	Post-filter fan-out
notify	S->C	in-app toast payloads	Mirrors push when app open

7.3 Validation Rules (server-side, always)

Velocity clamp (max sprint + 15%), teleport rejection (distance/time), zone legality (no actions from the wrong district/instance), rate ceilings per op, session-token binding on every crime/raid/fight message.

7.4 Latency Budget

Design tolerance: 250 ms RTT feels identical to 40 ms for everything except your own movement (client-side immediate) and minigame inputs (fully client-side theatre, \$5.5). This is the PvE dividend again — we buy Hetzner prices with it.

7.5 Binary Later, Maybe

If profiling at scale shows socket bandwidth mattering (it will not for a long time at 10 Hz interest-managed), switch pos/pos_batch to a packed binary schema. Do not spend week one on this.

8. Security & Anti-Cheat

8.1 Principles

1. Client is untrusted, always.
2. Money and progress mutate only inside server transactions.
3. Never hard-punish on heuristics alone — flag, floor, review.
4. Make cheating unprofitable rather than impossible.

8.2 Surfaces & Answers

Threat	Answer
Memory-edited client values	Cosmetic only; server recomputes everything (§5.3–5.5)
Replayed / forged RPCs	Session-bound single-use tokens (crime_sessions, raid ids), idempotency keys on claims
Speed/teleport hacks	\$7.3 clamps; violations soft-flag
Minigame bots	Duration windows + human-metric plausibility + perfection-rate anomaly detection -> shadow-flag queue
Economy exploits	Append-only ledger, nightly invariant audit, mint/sink anomaly alerts, market price bands
Multi-account feeding	Device-hash + behaviour correlation flags; market tax + price floors blunt the profit
DDoS / abuse	Caddy rate limits, per-account RPC ceilings, Hetzner network filtering; city chat rate limits

8.3 RNG Policy

All rolls server-side, seeded per session, seeds logged with outcomes. Big wins (jackpot tier, heist pools) are replayable from the log — disputes and "was that rigged?" both die in one query. Published odds (GDD §18.2) are read from the same config the roller uses: the honesty is structural.

8.4 Review Queue

Shadow-flags accumulate per account with reasons and evidence refs; weekly triage in the admin panel; actions from "watch" to wipe+ban. Target false-positive rate: zero visible punishments without human review.

9. Privacy, GDPR & UK Compliance

- **Data minimisation:** we hold display name, optional email, device token, gameplay data. No real names, no addresses, no payment data in this phase. IPs transient in logs (14 days).
- **Hosting in the EU** (Hetzner, Germany) with a signed DPA; UK players served from EU — adequacy holds; revisit only if that changes.
- **Rights pipeline:** in-app "download my data" (JSON export job) and "delete my account" -> erasure job: PII wiped, player row anonymised (Deleted Roadman #x), ledger/war history retained as anonymous aggregates (legitimate interest: economy integrity), chat authored by the account tombstoned.
- **Retention:** chat 30 days; raw telemetry 90 days then rollups; backups 30 days; report evidence 90 days.
- **Age gating:** 18+ self-declaration at first launch + store rating enforcement (PEGI 18 pathway per GDD §21). No child-directed processing, no ad SDKs in this phase — the privacy label stays clean.
- **Breach playbook** written before launch: detect (alerting §10.4) -> contain -> assess -> notify ICO/CNIL-equivalent within 72 h if thresholds met. Boring, mandatory, one page.

10. Infrastructure & DevOps

10.1 Environments

dev (local docker-compose) -> staging (small VPS, production-shaped, season-rollover rehearsals) -> prod. Config differences are environment variables only.

10.2 Launch Topology (prod)

One Hetzner CCX23 (4 vCPU dedicated / 16 GB): Caddy + Nakama + Postgres + Metabase via docker-compose. One Storage Box for backups + asset bundles. That is the whole empire on day one, and it is enough (§11).

10.3 CI/CD

GitHub repo; Actions pipeline: typecheck + unit tests (economy invariants, combat/crime resolvers as pure-function golden tests) -> build images -> deploy to staging on merge -> tagged deploy to prod. Claude Code drives the repo; the pipeline keeps it honest. Database migrations run as a gated pipeline step, never by hand.

10.4 Observability

- Metrics: Prometheus + Grafana (or Netdata to start): CPU, socket count, match count, RPC latency p95, DB connections, job lag, push delivery rates.
- Logs: structured JSON to Loki/journald; error alerting via Grafana -> your phone (Telegram bot is fine).
- Alarms that matter at launch: server down, error-rate spike, ledger audit mismatch, war/season job failure, push delivery collapse, disk > 80%.

10.5 Backups & Recovery

Nightly base backup + WAL streaming (pgBackRest) to the Storage Box; 30-day PITR; monthly restore drill on staging (a backup you have not restored is a rumour). RTO target 2 h, RPO target 5 min — generous and achievable solo.

11. Capacity Planning & Costs

11.1 Assumptions (soft launch)

DAU 5,000 · peak CCU ~10% = 500 · avg 35 players/district instance -> ~14 district instances + ~8 concurrent raid/heist instances · 10 Hz pos in/out interest-managed -> low-thousands msgs/s aggregate · RPC volume dominated by collects/crimes: tens/sec.

11.2 Fit

Nakama on 4 dedicated vCPUs handles this envelope with headroom (real-time load here is presence-light, not physics); Postgres at this scale is small (ledger growth ~ a few GB/month; partition telemetry). The single-node CCX23 topology holds to roughly 15–20k DAU before anything needs splitting; first splits: Postgres to its own node, then a second Nakama node behind Caddy.

11.3 Monthly Cost Table (launch)

Item	EUR / month
Hetzner CCX23 (prod)	~55
Hetzner CPX21 (staging)	~9
Storage Box 1 TB (backups/assets)	~4
Domain + misc	~5
FCM / APNs / Metabase / Nakama	0
Total	~75

Scale-up to ~20k DAU: roughly EUR 150–250/month. The backend bill stays a rounding error against art.

12. Backend Milestones (aligned to GDD §22.2)

M	GDD milestone	Backend deliverables	Definition of done
M0	The Snatch	None live. Client-local stub implementing the §5.5 interface (same payload shapes) so M1 swaps stub for server without touching game code	Stub interface frozen

M	GDD milestone	Backend deliverables	Definition of done
M1	Vertical Slice	Auth/accounts, players/stats, regen engine, ledger + wash, crime resolution RPCs, config-as-data v1, telemetry ingest + Metabase retention/funnel boards, push tokens + basic sends	50-tester closed test runs entirely server-authoritative; D1 measured from <i>our</i> dashboard
M2	The World Pushes Back	District matches + presence + interest mgmt, NPC sim, beef, combat resolver, crew v1, street events, sink-ratio dashboard	Two phones see each other move; all fights server-logged
M3	Take the Postcode	Raid instances + snapshots + determinism, territory + control points, defence layouts, lines + runs + convoy encounters, scouting, Feed v1	Solo player takes a postcode end-to-end with zero client-trusted writes
M4	Firms & War	Groups=Firms, firm bank, chat + moderation chain, war state machine + jobs + tickets + stripes, live-defence prompt path, leaderboards, notification matrix enforcement	One full 48 h war resolves by job with correct transfers, shields, ceremonies
M5	Season 1 Launch	Seasons + rollover + soft map reset, marketplace escrow, admin panel + review queues, GDPR export/erasure jobs, load test to 3x target CCU, backup/restore drill, breach playbook	Three green weeks on \$22.3 KPIs from production dashboards

13. Open Questions (need your call)

1. **Client engine: Godot 4 vs Unity** — needed before M1 (SDK choice, §4.3). Recommendation on record: Godot.
2. **Nakama vs Colyseus final sign-off** — recommendation on record: Nakama; decide after a 2–3 day M1 spike if you want to feel both.
3. **Analytics:** Metabase-on-SQL (recommended, zero cost) vs PostHog self-hosted (nicer funnels, heavier op).
4. **Region strategy:** single EU region indefinitely (recommended for years) vs UK edge later.
5. **Binary protocol timing:** parked until profiling demands it (§7.5) — confirm you are happy parking it.

Appendix A — Sample Payloads

```
// crime_start -> response
{ "session_id": "cs_9f2", "type": "phone_snatch", "seed_ref": "s_a41",
  "params": { "mark_band": [40, 120], "window_ms": 900, "min_ms": 6000, "max_ms": 45000 } }

// crime_finish -> result (drives the reveal ceremony)
{ "outcome": "crit", "loot": [ { "cash_dirty": 118 }, { "item": "phone_flagship", "rarity": "peng" } ],
  "xp": { "track": "snatch", "gain": 10, "level_up": false },
  "heat": +1, "near_miss": { "fired": true, "fed_margin_ms": 2800 } }

// defend_prompt (push + socket)
{ "raid_id": "r_771", "attacker": "M4NDEM", "window_ends": 1753371045, "deep_link": "ends://raid/r_771" }
```

Appendix B — Config-as-Data Sample (crime table row)

```
{ "key": "crimes.phone_snatch", "version": 7,
  "body": { "rank_req": 1, "nerve": 2, "payout": [40, 120],
    "crit": { "odds": 0.08, "item_pool": "phones_t1" },
    "jackpot_odds": 0.02, "xp": 10, "heat": 1,
    "districts": ["strip", "neon_row"], "pity_window": 25 } }
```

Appendix C — Golden-Test List (pure functions to lock first)

Regen math (§5.3) · ledger invariants (§5.4) · crime roller incl. pity floor (§5.5) · combat resolver vs GDD §10 examples · flee formula bounds · raid stash cap · war stripes tally + resolution transfers · wash fee tiers · notification cap/quiet-hours gate. These tests are the contract Claude Code builds against — write them before the implementations.

ENDS Backend Design Document v1.0 — written for Snicla. Companion: ENDS GDD v1.1. Next deliverable when you are ready: the M1 build spec (repo layout, module skeletons, and the golden tests from Appendix C) handed straight to Claude Code.